# From Raw FIT files to Explainable AI:

## Reconstructing real training behavior from raw data — and linking it to physiological adaptation

## Introduction — From Averages to Physiology

Most training platforms summarize performance using averages.

Average power. Average heart rate. Normalized power.

These metrics are easy to interpret, but they hide the underlying structure of training. A three-hour ride can look like a clean Zone 2 session on paper, while in reality it consists of fragmented efforts, surges, and inconsistent intensity.

From a physiological perspective, these differences matter. Training adaptations are driven by **specific stimuli**, not by averages.

This project started from a simple question:

**Can we reconstruct what an athlete actually did — and link it to how the body adapts?**

To answer that, I built an end-to-end data pipeline based on **880 .FIT files of an avid cyclist** transforming raw sensor data into:

- structured interval classifications
- physiologically meaningful features
- and an explainable machine learning model

The goal was not just to predict performance, but to understand **which training patterns actually drive adaptation**.

# System Overview - From Raw Data to Insight

Before diving into the individual components, the diagram below summarizes the full data pipeline.

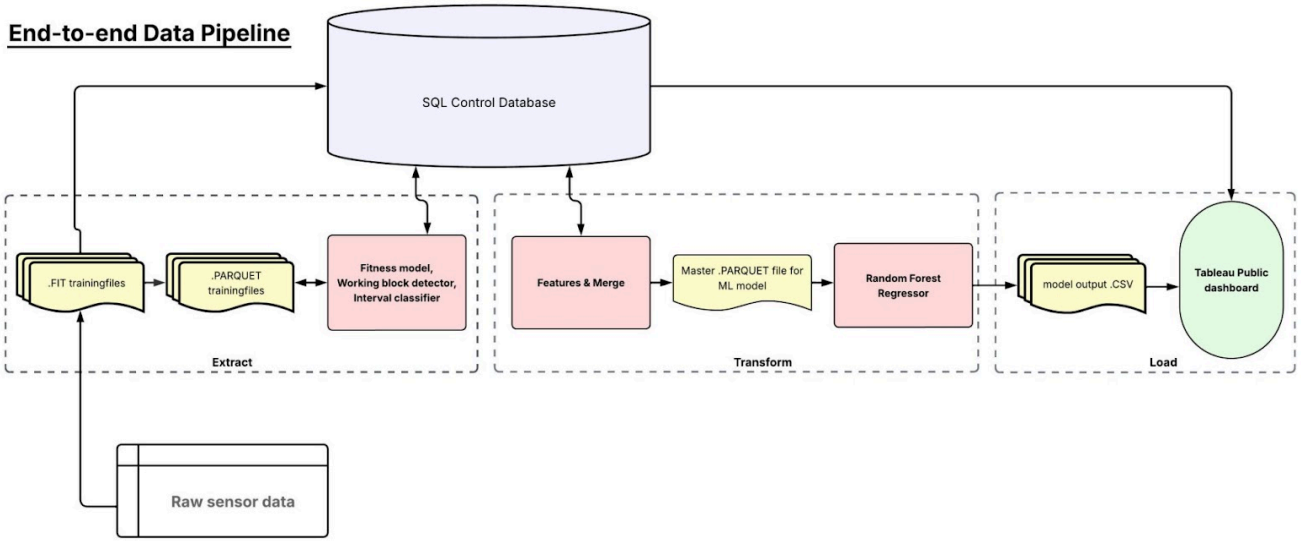


Fig. 1: Data Pipeline Chart

Introduction — From Averages to Physiology
System Overview - From Raw Data to Insight
Ch. 1: The "Why" – Beyond the Average
- The Z2 Fallacy
- The Coach's Dilemma
- Clicking into Place: Python & AI

Ch. 2: Data Chaos & Architecture (The Foundation)
- Taming the Hertz: From .FIT to .Parquet
- The Audit: Kudos and Consistency
- The "Bridge Sprint" Discovery
- The Workaround: eTRIMP

Ch. 3: Building the "Interval Classifier" - The Core Pipeline
- Script 1: Detecting working blocks
- Script 2 — The Interval Classifier
- The Interval Dictionary
- Scale of the Interval Dataset
- Handling Micro-Coasting
- Building the Interval Database

Ch. 4: Physiological Feature Engineering (The Magic)
- The Question: What Does "Better" Mean?
  - Dimension 1 — Aerobic Base Development
  - Dimension 2 — Efficiency at Race-Relevant Intensities
- Introducing Time: Training as a Cumulative Process
- Additional Contextual Features
- Building the Feature Dataset

Ch. 5: The Machine Learning Engine (Predictive Analytics)
Ch. 6: The Deliverable & Conclusion (The Dashboard)
- 1. Training Load & Execution
- 2. Model Insights — Feature Importance & SHAP Values
- 3. Aerobic Decoupling — Trend Validation
  - Model validation (Aerobic Decoupling)
- 4. Efficiency Factor — Longitudinal Analysis & Simulation
  - Phase 1 — Breakdown
  - Phase 2 — Alignment
  - Model validation (Efficiency Factor)
  - The What-If Simulator

Ch. 7: Limitations and road ahead (v2.0)
Conclusion & Impact
Final Word

# Ch. 1: The "Why" – Beyond the Average

Most endurance athletes follow the same post-workout ritual. The ride ends, Garmin Connect or Strava opens, and the familiar graphs appear: power, heart rate, cadence. The numbers look reassuringly precise.

But the question remains: how much do they actually tell us?

Training analysis is still largely built around summary statistics. Average Power, Average Heart Rate, and Normalized Power are calculated over the entire duration of a session and used as proxies for training quality. Over time, athletes and coaches develop an intuition for interpreting these values. If the averages improve, we assume the athlete is progressing.

Yet averages describe outcomes, not mechanisms.

The central question is not simply whether a training session produced fatigue or fitness, but whether it induced the **specific physiological adaptation** it was designed to target—and whether it did so efficiently.

## The Z2 Fallacy

Consider a common example: a three-hour Zone 2 ride.

The ride finishes and the average power falls neatly inside the Zone 2 range. On paper, the session appears successful.

However, averages conceal variability. A ride that oscillates between easy pedaling, small surges, and brief threshold efforts can easily produce the same average power as a perfectly controlled aerobic session.

Physiologically, these two sessions are not equivalent.

A true Zone 2 stimulus targets mitochondrial density, improves capillarization in Type I muscle fibers, and enhances lactate clearance capacity. These adaptations require a sustained metabolic environment. If intensity repeatedly drifts outside the intended range, the metabolic signal becomes diluted.

In other words, the athlete may complete a three-hour ride with a Zone 2 average while spending far less time producing a meaningful Zone 2 stimulus.

In high-performance sport, this distinction matters. When a session is prescribed as Zone 2, the physiological objective is precise. If the stimulus deviates substantially from that target, the training signal becomes noise.

## The Coach’s Dilemma

During my years working with athletes, I tried to control every variable that could reasonably be measured.

Baseline lactate profiles were mapped across different wattages. Training sessions were scheduled around life stress and recovery patterns. Biomechanics were monitored carefully. By conventional standards, the data looked clean and structured.

And yet, the training outcomes did not always align with expectations.

Performance sometimes improved faster than predicted. At other times it stagnated despite apparently well-executed training.

Situations like these are deeply frustrating for a coach. Not because progress is absent, but because the **mechanism behind the result remains unclear**.

I have always had a low tolerance for unexplained outcomes. In physiology, as in statistics, patterns rarely appear without underlying structure. When the output does not match the expected input, the explanation is usually not randomness—it is that our analytical lens lacks sufficient resolution.

## Clicking into Place: Python & AI

My interest in sports physiology has always been tightly linked to data. However, it was only when I began working with Python and machine learning models that a different perspective emerged.

Most commercial training platforms operate at a relatively coarse level of analysis. They summarize sessions well, but they struggle to capture the complex micro-structure of real training data: fluctuating power profiles, irregular interval patterns, and individual physiological responses.

The result is a system optimized for convenience rather than depth.

This realization led to a clear objective: build an end-to-end data pipeline capable of transforming raw training files into **explainable physiological insight**.

The goal is not simply to generate another fatigue score or recovery indicator.

I believe that athletes need —and maybe coaches even more— an explanation grounded in their own data.
Why did this session produce fatigue?
Which physiological systems were actually stressed?
And how closely did the executed workout match the intended training stimulus?

Without answering these questions, training analysis remains an exercise in approximation. With them, it becomes a structured investigation into how performance is actually built.

# Ch. 2: Data Chaos & Architecture (The Foundation)

Before building an analytical model, the underlying data structure has to be reliable. In other words: before building a brain, you need a skeleton.

For this project, a former athlete I coached shared 2 years of his training history: **880 `.FIT` files**, spanning February 2024 to February 2026.

At first glance, it is simply a folder of workout files. From a data engineering perspective, it is a dense time-series dataset containing millions of high-frequency sensor observations.

## Taming the Hertz: From .FIT to .Parquet

`.FIT` files are binary encoded and highly efficient for storage, but they are not designed for large-scale analytical workflows.

Each file records data at **1 Hz resolution**—one row per second of activity. The implications are significant:

- A 4-hour endurance ride produces roughly **14,400 rows**
- A 7-hour long ride produces **25,000+ rows**
- Across nearly **900 files**, the dataset quickly grows to **millions of observations**

Converting everything to CSV would have been straightforward but inefficient. CSV is slow to read, memory-heavy, and poorly suited for column-oriented analytics.

Instead, I converted all parsed `.FIT` data into **Apache Parquet**, a columnar storage format optimized for analytical workloads. Combined with **Pandas**, this allowed for fast filtering, selective column reads, and efficient aggregation across the entire dataset.

To organize the system, I built a **SQL metadata table** that acts as the index layer of the pipeline. Each activity is tracked with:

- file identifiers
- timestamps
- activity type
- sensor availability
- integrity flags for missing streams
- Other key training metrics

This structure made it possible to audit the dataset with precision and detect inconsistencies before any modeling began.

## The Audit: Kudos and Consistency

Machine learning systems are only as reliable as the data they ingest. Before any modeling could happen, the entire dataset needed to be audited.

The initial SQL inspection revealed the composition of the training archive:

- **176 strength sessions**
- **245 indoor sessions (Zwift)**
- **456 outdoor rides**

From a physiological perspective, these categories are not interchangeable. Strength work alters neuromuscular fatigue and recovery dynamics, while indoor rides tend to produce cleaner and more controlled data.

Outdoor rides, however, introduced a new layer of complexity.

Out of the **456 outdoor rides**, **96 sessions contained no power data**. Most of these were daily commutes: roughly 26 km in the morning and another 26 km in the evening.

For a conventional training analysis system, these sessions would likely be classified as incomplete or discarded entirely. From a data perspective, they looked like missing information.

But physiology rarely leaves patterns unexplained.

```sql
SELECT startuur, avg_temperature, total_distance, avg_speed, avg_heart_rate, max_heart_rate
FROM Ritten
WHERE avg_power IS NULL
AND activity_type IS 'WIELRENNEN';
```

| | startuur | avg_temperature | total_distance | avg_speed | avg_heart_rate | max_heart_rate |
|---|---|---|---|---|---|---|
| 1 | 06:03:38 | -1 | 26106.91 | 28.6416 | 143 | 169 |
| 2 | 08:06:26 | 7 | 102237.74 | 28.4616 | 149 | 176 |
| 3 | 17:17:52 | 7 | 40253.17 | 27.7308 | 140 | 164 |
| 4 | 16:40:36 | 7 | 26648.66 | 28.3644 | 142 | 170 |
| 5 | 16:50:50 | 7 | 25992.1 | 30.0744 | 143 | 172 |
| 6 | 11:02:07 | 10 | 39099.83 | 27.2628 | 144 | 168 |
| 7 | 06:17:01 | 4 | 39669.18 | 28.3464 | 142 | 168 |
| 8 | 16:37:28 | -2 | 25947.6 | 26.82 | 138 | 158 |
| 9 | 19:17:54 | 5 | 28108.26 | 30.456 | 148 | 178 |
| 10 | 07:10:44 | -1 | 26261.77 | 29.394 | 141 | 162 |
| 11 | 08:14:44 | 10 | 103090.54 | 28.782 | 147 | 172 |
| 12 | 06:06:44 | 3 | 26282.01 | 28.9548 | 146 | 176 |
| 13 | 06:05:54 | 10 | 26108.35 | 29.8656 | 145 | 169 |
| 14 | 16:40:40 | 11 | 26355.38 | 30.7944 | 152 | 189 |
| 15 | 17:06:36 | 5 | 25978.43 | 29.7 | 139 | 156 |
| 16 | 15:13:21 | 0 | 73681.13 | 29.214 | 148 | 178 |
| 17 | 16:46:32 | 5 | 26435.3 | 28.0612 | 131 | 146 |
| 18 | 16:43:03 | 10 | 26038.35 | 30.96 | 148 | 173 |
| 19 | 06:48:57 | 4 | 26144.91 | 29.7468 | 154 | 180 |
| 20 | 13:47:21 | -1 | 45500.99 | 31.3344 | 154 | 182 |
| 21 | 12:06:32 | 9 | 28298.2 | 27.882 | 133 | 161 |
| 22 | 06:33:37 | 8 | 22503.84 | 28.3356 | 156 | 180 |
| 23 | 05:59:52 | 3 | 39824.33 | 28.0944 | 145 | 167 |
| 24 | 17:49:08 | 5 | 26600.56 | 24.8256 | 135 | 156 |
| 25 | 07:17:44 | 10 | 26123.84 | 28.9368 | 146 | 184 |

Fig. 2: SQL snippet illustrating the bridge sprint discovery.

## The "Bridge Sprint" Discovery

When I examined the heart rate traces of these commuting rides, a different pattern emerged.

Heart rate repeatedly spiked into **high Zone 4 and low Zone 5**—not once, but consistently across many of these sessions (fig 1.).

This was unexpected for rides that were assumed to be low-intensity transportation.

After asking the athlete about it, the explanation was simple:

“Bridge sprints. I need them to feel good.”

What initially appeared to be incomplete data turned out to be a consistent behavioral pattern. These commutes were not passive rides. They contained repeated anaerobic efforts embedded within otherwise moderate rides.

From a training perspective, this had real implications. These short but frequent high-intensity spikes introduced an additional anaerobic stimulus that was not accounted for in the planned training structure. In effect, they were quietly interfering with the aerobic adaptations we were trying to develop.

This was the same problem introduced in Chapter 1: **averages hiding physiological reality**.

### The Workaround: eTRIMP

The missing power data created a second problem. Without power metrics, it was impossible to compute **Training Stress Score (TSS)** for these rides using the standard power-based method.

Ignoring 96 sessions would severely distort the athlete's **Chronic Training Load (CTL)**.

To close the gap, I implemented a proxy load metric based on **Edward's TRIMP (eTRIMP)**.

The method assigns a weight to each second spent in a five-zone heart rate model:

- Zone 1 → 1 point per second
- Zone 2 → 2 points per second
- Zone 3 → 3 points per second
- Zone 4 → 4 points per second
- Zone 5 → 5 points per second

Aggregating these weighted seconds produced a heart-rate-based training load estimate for every ride lacking power data.

When validated against Garmin's native TSS on rides where both power and heart rate were available, the proxy showed **strong and significant correlation**, making it a reliable substitute.

The result was a **complete, gapless load timeline** across the entire two-year dataset.

With the data architecture stabilized and missing signals reconstructed, the pipeline was finally ready for engineering the features.

## Ch. 3: Building the "Interval Classifier" - The Core Pipeline

With the data architecture in place, the next step was to transform raw time-series data into physiologically meaningful units.

Training files contain continuous streams of numbers—power, cadence, heart rate—but meaningful analysis requires segmentation. The goal of this stage was therefore simple in principle but complex in execution: **detect and classify every meaningful effort within every ride.**

For this part of the project, I developed two Python scripts forming the core of the pipeline. To ensure reliability, I restricted the dataset to rides where **power data was available**. This resulted in **505 rides** used for interval analysis (245 indoor Zwift rides and 360 outdoor rides

## Script 1: Detecting working blocks

For each activity, the pipeline retrieved the file name and the athlete's **Functional Threshold Power (FTP)** at the time of the workout from the SQL database. The corresponding `.parquet` file was then loaded into a Pandas dataframe.

To detect physiologically meaningful effort, I calculated the athlete's **aerobic threshold** as:

**0.78 × FTP**

This value was intentionally chosen on the upper side of the aerobic continuum to ensure that only metabolically relevant work segments were captured.

Before classification, the power stream was smoothed using a **centered 5-second rolling average**. Cycling power data often contains short spikes caused by terrain changes or gear shifts, and this smoothing step acts as a buffer against such transient artifacts.

The script then created a Boolean column: 'is_working_block'

Each second received a value of **1** if the smoothed power exceeded the aerobic threshold and **0** otherwise. This transformed a continuous power stream into a binary signal representing active physiological work.

The modified dataset was written back to the `.parquet` file, ready for the next stage.

## Script 2 — The Interval Classifier

The second script forms the central component of the project: the **interval classifier**.

Rather than modifying the `.parquet` files, this stage extracts structured interval data and stores it in a new **SQL interval table**.

The first step was to detect contiguous segments of working and non-working activity. Using Pandas, intervals were grouped with the expression:

```
df['is_working_block'].diff().ne(0).cumsum()
```

This technique assigns a unique identifier to each continuous block where the Boolean state remains constant. From these grouped segments, the pipeline computed a set of metrics including:

- interval duration (seconds)
- average power
- average heart rate
- average cadence
- interval start time

Additional derived metrics were also calculated, such as:

- **Normalized Power**
- **Aerobic decoupling**

These features created a detailed physiological profile for each detected interval.

## The Interval Dictionary

The central innovation of the classifier lies in the **interval dictionary** used to label each detected segment. I defined **17 interval categories**, representing well-known structures in endurance training, including:

- Tabata micro intervals
- $VO_2$ micro intervals
- Long $VO_2$ intervals
- Sweet Spot
- Tempo
- Threshold

Each category was defined by explicit ranges for:

- power relative to FTP
- interval duration
- rest duration
- cadence characteristics

These intervals represent what I call **“clean intervals”**—structured efforts typically prescribed by a coach.

However, real-world training rarely follows textbook structures. Much of the true complexity lies in the **unstructured efforts that appear between them**.

To capture this, the classifier also included several **surge categories**, such as:

- Threshold surges
- Sweet Spot surges
- Anaerobic surges

These represent the small intensity spikes that often occur during outdoor riding.

Athletes know these moments well: testing the legs on a recovery ride, pushing slightly harder over a bridge, or briefly riding above threshold to reconnect with a group.

These efforts often disappear when looking at ride averages. Even variability indices can remain within acceptable ranges. Yet physiologically they represent **distinct metabolic demands**.

By explicitly classifying them, the model provides a far more detailed representation of what the athlete actually did during a ride.

## Scale of the Interval Dataset

After the first full runs of the pipeline and several refinements to the interval dictionary, the classifier produced approximately:

**72,000 detected intervals**

Only a small fraction—roughly **300 segments**—remained in a generic fallback category, indicating that the classification system captured the vast majority of training patterns.

## Handling Micro-Coasting

During initial testing, another artifact appeared in the segmentation process.
Short drops below the aerobic threshold frequently appeared inside otherwise continuous working intervals. These were typically brief coasting moments lasting only a few seconds, often caused by traffic, terrain, or road conditions.

From a physiological perspective, these interruptions should not split an interval into multiple segments.

To address this, I introduced a **micro-coast buffer** before the interval detection stage.

All segments shorter than 15 seconds that were classified as non-working were reassigned as working blocks. Several thresholds were tested between 5 and 15 seconds, and 15 seconds produced the most realistic interval reconstruction when visually inspecting the rides.

```python
def detect_intervals(df):
    # ---- Micro-coast buffer ----
    df['temp_id'] = df['is_werkblok'].diff().ne(0).cumsum()
    block_sizes = df.groupby('temp_id').size()
    block_status = df.groupby('temp_id')['is_werkblok'].first()

    short_rest = (block_status == 0) & (block_sizes <= 15)

    df.loc[df['temp_id'].isin(short_rest[short_rest].index), 'is_werkblok'] = 1

    # ---- After buffer detector ----
    df['interval_id'] = df['is_werkblok'].diff().ne(0).cumsum()

    all_blocks = df.groupby('interval_id').agg(
        status = ('is_werkblok', 'first'),
        duur = ('is_werkblok', 'count'),
        cadence = ('cadence', 'mean'),
        avg_hr = ('heart_rate', 'mean'),
        avg_power = ('power', 'mean'),
        starttijd = ('timestamp', 'first')
    )

    all_blocks['rest'] = all_blocks['duur'].shift(-1)

    custom_metrics = df.groupby('interval_id').apply(lambda x: pd.Series({
        'np_power': get_np(x['power']),
        'decoupling': calc_decoup(x),
    }))

    all_intervals = pd.concat([all_blocks, custom_metrics], axis=1)
    all_intervals['EF'] = all_intervals['np_power'] / all_intervals['avg_hr']
    all_intervals['source_file'] = df['source_file'].iloc[0]
    all_intervals['rest'] = all_intervals['rest'].fillna(0)

    return all_intervals
```

Fig. 3: Script modification for the micro-coast buffer in the detect intervals function

This adjustment reduced the interval count from approximately 72,000 to 42,000 segments, producing a segmentation that more accurately reflected real training structure.

At this stage, the threshold was selected based on **domain knowledge** and **visual inspection.**
However, to verify that this choice was not arbitrary, I performed an additional **sensitivity analysis.**

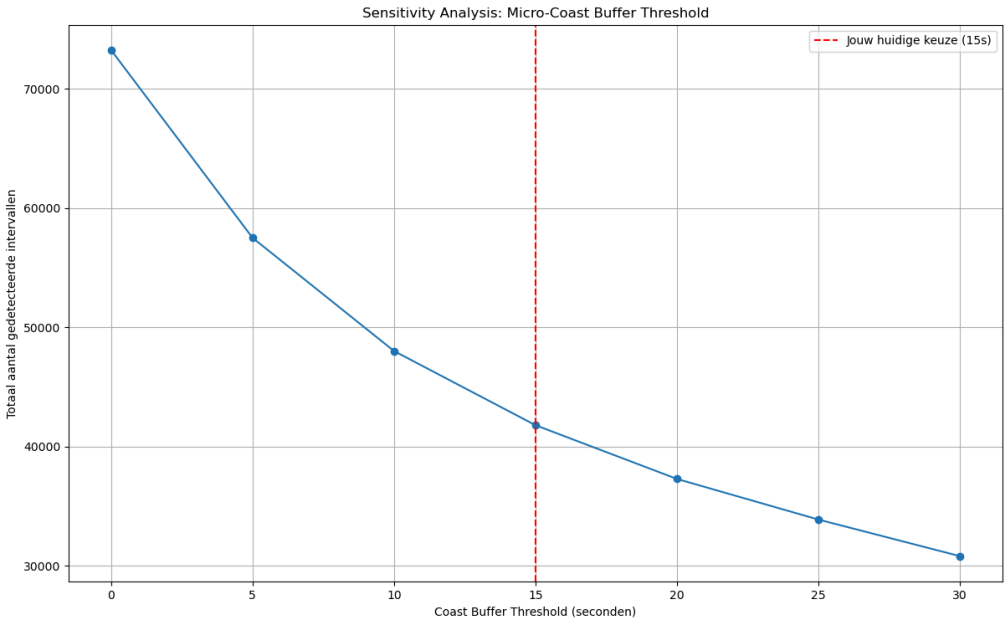


Fig. 4: Plot of the first sensitivity analysis.

A global analysis showed how the total number of detected intervals evolves as the buffer increases from 0 to 30 seconds. As expected, larger buffers merge more intervals, but the effect is gradual — there is no abrupt structural collapse. This indicates that the segmentation is stable and not overly sensitive to the parameter choice.

In parallel, I **manually inspected** individual workouts and reconstructed intervals using different buffer thresholds (**5s to 30s**). This revealed two consistent patterns:

- Small buffers (e.g. 5s) sometimes split clean intervals due to brief power drops
- Very large buffers (e.g. 30s) could merge intervals that should remain distinct

In practice, these edge cases were limited. Most intervals remained consistent across buffer values, confirming the robustness of the detection method.

The **final choice** of a 15-second buffer therefore represents a balance:
tolerant enough to absorb micro-coasting, while still preserving the structure of the workout.

More importantly, this analysis highlights a broader point.

When intervals appear fragmented, this is often not a limitation of the model, but a reflection of execution. Well-paced efforts produce clean, continuous intervals. Fragmented power profiles do not.

In that sense, the model does **not smooth out training noise — it exposes it.**

| | A | B | C | D | E | F | G | H | I | J | K | L | M |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Bestand | Start (GT) | Eind (GT) | (Buffer 5s) | (Buffer 10s) | (Buffer 11s) | (Buffer 12s) | (Buffer 13s) | (Buffer 14s) | (Buffer 15s) | (Buffer 20s) | (Buffer 25s) | (Buffer 30s) |
| 2 | tp-3396952. | 01:36:31 | 01:40:33 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 3 | tp-3396952. | 01:45:33 | 01:49:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 4 | tp-3396952. | 01:56:11 | 02:00:14 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 5 | tp-3396952. | 01:45:55 | 01:46:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 6 | tp-3396952. | 01:47:15 | 01:47:55 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 7 | tp-3396952. | 01:48:35 | 01:49:15 | 2 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 8 | tp-3396952. | 01:49:55 | 01:50:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9 | tp-3396952. | 01:51:15 | 01:51:55 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 10 | tp-3396952. | 01:54:35 | 01:55:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 11 | tp-3396952. | 01:58:35 | 01:59:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 12 | tp-3396952. | 02:02:35 | 02:03:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 13 | tp-3396952. | 02:06:35 | 02:07:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 14 | tp-3396952. | 02:10:35 | 02:11:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 15 | tp-3396952. | 02:14:35 | 02:15:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 16 | tp-3396952. | 02:25:35 | 02:29:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 17 | tp-3396952. | 02:33:35 | 02:37:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 18 | tp-3396952. | 02:41:35 | 02:45:35 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 1 |
| 19 | tp-3396952. | 02:49:35 | 02:53:35 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

Fig. 5: Manual inspection of some of the intervals

## Building the Interval Database

The interval detection pipeline processes the dataset **file by file**.

For each `.parquet` file:

1. The ride is loaded into a Pandas dataframe.
2. All working and non-working segments are detected.
3. Interval metrics are computed and each segment is classified using the interval dictionary.

The intervals generated from a single ride are first stored in a **temporary list**.

After processing that file, this list is **appended to a master list** that lives outside the processing loop. The pipeline then proceeds to the next file and repeats the process.

Only after **all files have been processed** are the accumulated interval lists concatenated into a single dataframe. This dataframe—containing roughly **42,000 classified intervals**—is then written to a dedicated **SQL interval table**.

Structuring the pipeline in this way avoided repeated database writes during processing and keeps the workflow computationally efficient while still producing a fully structured interval dataset.

# Ch. 4: Physiological Feature Engineering (The Magic)

With both the **interval table (micro structure)** and the **calendar table (macro structure)** in place, the next step was to design the features that would feed the machine learning model.

Just as engines require fuel, machine learning models require structured patterns. Raw training data—power streams, heart rate traces, timestamps—contains enormous amounts

of information, but very little structure. Feature engineering is the process of transforming those signals into variables that capture the underlying physiology.

This is where my background as both an athlete and coach became essential.

## The Question: What Does “Better” Mean?

My coaching philosophy was heavily shaped by my own experience as a high-level athlete.

Before working with a coach, my training strategy was simple: go hard in almost every session. When I finally started structured training, I assumed the program would simply require going even harder.

Looking back fifteen years later, I realize how unsustainable that approach was. Most sessions were effectively **Zone 4 or Zone 5 efforts**, leaving little room for systematic adaptation.

The introduction of structured aerobic work—particularly **Zone 2 training**—forced a deeper question:

*Why do certain training stimuli make an athlete better?*

At its core, training is a repeated cycle of **stress and adaptation**. Each interval targets a specific physiological system, and over time the accumulation of these stimuli shapes the athlete’s energy systems to match the demands of competition.

But translating that principle into measurable variables is not trivial.

For the purposes of this project, I divided “getting better” into two distinct dimensions.

## Dimension 1 — Aerobic Base Development

The first dimension represents **general aerobic fitness**, the foundation on which all endurance performance is built.

To quantify this, I focused on **aerobic decoupling** during sustained Zone 2 efforts.

Aerobic decoupling measures the relationship between power output and heart rate over time. It is calculated by comparing the ratio between normalized power and heart rate in the **first half** of a steady endurance block with the **second half** of that block.

In practice:

- Endurance blocks were required to last **at least 40 minutes**
- Ideally longer durations were used when available

Under stable conditions, well-trained athletes typically show **less than 5% cardiac drift**. When decoupling exceeds this threshold, it often indicates fatigue, insufficient aerobic conditioning, or environmental stress.

By tracking this metric across training sessions, it becomes possible to observe whether the athlete's aerobic system is becoming more efficient over time.

## Dimension 2 — Efficiency at Race-Relevant Intensities

The second dimension represents performance at **race-specific intensities**.

Here, I focused on the **Efficiency Factor (EF)**, defined as:

```
Efficiency Factor = Normalized Power / Heart Rate
```

This metric was calculated only for intervals that occur in intensity domains where physiological steady states are achievable:

- Tempo
- Sweet Spot
- Threshold

These intensity zones are metabolically stable enough to produce meaningful efficiency measurements. Both lactate production and clearance approach equilibrium, and heart rate tends to stabilize, making the power-to-heart-rate relationship interpretable.

Higher EF values over time generally indicate improved cardiovascular and metabolic efficiency.

## Introducing Time: Training as a Cumulative Process

Training adaptations do not occur in isolation. Every session contributes to a larger timeline of accumulated stress and recovery.

Capturing this **temporal dimension** was therefore essential.

The feature engineering script loaded the interval table from SQL and grouped all intervals by training day. Because aerobic decoupling and efficiency factor had already been calculated at the interval level, the script aggregated these values to produce **daily physiological indicators**.

To capture adaptation across different time scales, I applied **Exponentially Weighted Moving Averages (EWMA)** with three half-life parameters:

- **7 days** → microcycle dynamics
- **28 days** → mesocycle dynamics
- **42 days** → longer-term adaptation trends

These smoothed indicators allowed the model to capture both short-term fluctuations and longer-term performance development.

Using EWMA also made it possible to construct a **continuous training calendar**. Rest days and low-intensity days—where no intervals exceeded the aerobic threshold—were still represented within the timeline, preventing artificial discontinuities in the dataset.

```python
# ------------------------------------------------------------------------------------------
# ------------------------------    FEATURE ENG: EWMAs    ------------------------------
# ------------------------------------------------------------------------------------------
print("\nCalculating EWMAs on intervals...")

# Aggregate per day
df = df_load.groupby(['datum', 'interval_label'])['duur'].sum().unstack(fill_value=0)
df = df.asfreq('D').fillna(0).reset_index()
df.columns.name = None

# Calculate EWMA's
ewma_cols = [col for col in df.columns if col != 'datum']
for col in ewma_cols:
    df[f"{col}_7d_ewma"] = df[col].ewm(halflife=7, adjust=False).mean().shift(1).fillna(0)
    df[f"{col}_28d_ewma"] = df[col].ewm(halflife=28, adjust=False).mean().shift(1).fillna(0)
    df[f"{col}_42d_ewma"] = df[col].ewm(halflife=42, adjust=False).mean().shift(1).fillna(0)
```

Fig. 6: Feature engineering of the EWMA's

## Additional Contextual Features

Several additional contextual features were incorporated to enrich the dataset:

- **Training start hour**, capturing potential circadian effects
- **Ambient temperature**, reflecting environmental stress
- **ATL (Acute Training Load),** calculated using the previously derived eTRIMP values
- **CTL (Chronic Training Load)**, calculated using the previously derived eTRIMP values
- **Strength training status**, indicating whether resistance training occurred the previous day

Together, these variables created a multi-layered representation of the athlete's training environment.

## Building the Feature Dataset

After all features were engineered, the results were written back to the database.

The final dataset was stored as a new SQL table, created by joining:

- the **calendar table** (macro timeline)
- the **interval dataset** (micro training structure)

This produced a unified training dataset combining physiological metrics, interval classifications, and contextual variables across the entire two-year period.

At this point, the raw training files had been transformed into a structured **physiological feature space**—a representation suitable for modeling performance adaptation with machine learning.

# Ch. 5: The Machine Learning Engine (Predictive Analytics)

The final step of the SQL-Python loop was identifying the key determinants of both **aerobic decoupling** and **efficiency factor**.

The dataset is **tabular**, contains **non-linear interactions**, and is noisy, with occasional extreme values. For this reason, I chose a **Random Forest Regressor**. While I initially considered a neural network, I rejected it for two reasons:

1. The small dataset size would likely cause **overfitting**.
2. My goal was **explainability** — to understand feature importance and compare it against physiological intuition. I wanted to break open the black box, not create another one.

The dataset was split into two segments:

- **Aerobic decoupling:** 22 days over two years with pure Z2 bouts (≥40 min, after excluding the 15' warm-up)
- **Efficiency factor:** 271 days with valid tempo, sweetspot, or threshold intervals

Because of the EWMA calculations, no data was lost and every day was represented in the temporal context.

After training the model, the initial **feature importance** and **SHAP values** were highly promising and aligned with physiological expectations. In addition, running **traditional linear statistics** on the top 10 features for both aerobic decoupling and efficiency factor yielded strong $R^2$ values, many statistically significant.

For detailed insights and interactive visualizations, refer to **Chapter 6**, where the Tableau dashboard brings the results to life

# Ch. 6: The Deliverable & Conclusion (The Dashboard)

The final output of the project is an interactive **Tableau dashboard**, structured across four analytical layers. Each layer translates the underlying data pipeline and machine learning model into interpretable insights.

## 1. Training Load & Execution

The first dashboard establishes the foundation by comparing **eTRIMP-based CTL** with traditional **power-based CTL**.

A clear divergence appears during winter periods, where training is often performed indoors (Zwift) or without power data. In these cases, traditional TSS underestimates training load, while eTRIMP maintains a consistent representation of physiological stress.

Below the main plot, two complementary views provide deeper insight:

- A **tree map** showing the distribution of classified intervals
- A **scatter plot** visualizing all detected intervals by duration and power

Interactive filtering allows inspection of individual interval types, revealing:

- average duration
- average power
- recovery structure

The interval distribution highlights an important pattern: a large proportion of training time is spent in **low-intensity and surge-type efforts**. For example:

- Z2 endurance intervals average ~120 seconds
- Z1 coasting intervals occur frequently with low power output

These segments contribute to total volume but often lack a strong physiological stimulus.

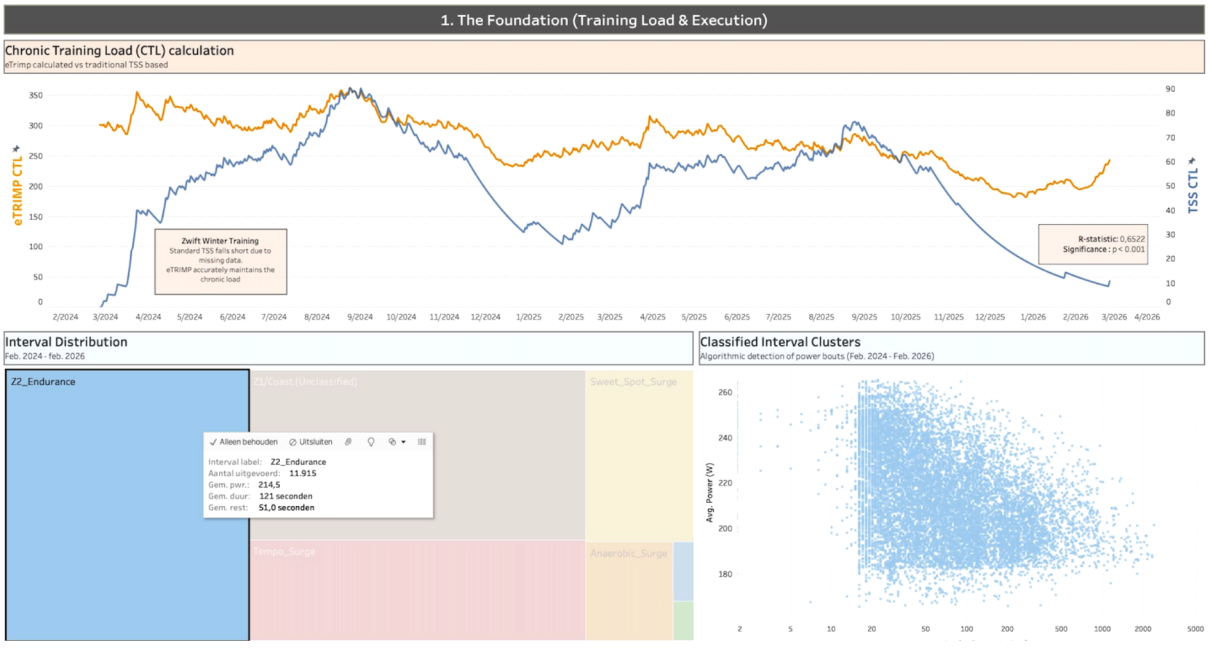


Fig. 7: Training load and execution layer within the Tableau dashboard

## 2. Model Insights — Feature Importance & SHAP Values

The second dashboard presents the output of the machine learning model.

Despite the dominance of low-intensity and surge intervals in total training volume, these features have **minimal importance** in predicting both:

- aerobic decoupling
- efficiency factor

Instead, the model highlights a smaller subset of features that align closely with physiological theory.

In this case study, the athlete has a naturally **anaerobic profile** — explosive, responsive to short efforts, and inclined toward high-intensity surges.

However, for long endurance events, performance depends on:

- improving **aerobic capacity ($VO_2$max)**
- reducing reliance on **anaerobic contribution (VLaMax)**

The model reflects this precisely.

One of the most interesting findings is the role of **time-dependent features**:

- **Tempo (42-day EWMA)** → strong positive effect on efficiency
- **Tempo (28-day EWMA)** → negative effect

This apparent contradiction is physiologically meaningful.

Longer-term tempo work builds the aerobic engine, while excessive medium-term load introduces **muscular fatigue** and reduces efficiency in race-specific efforts.

In practice, this translates directly to training strategy:

- build aerobic capacity over longer periods
- reduce fatigue closer to race day
- reintroduce intensity selectively ($VO_2$max, anaerobic efforts)

This is classical periodization — but now **quantified in the data**.

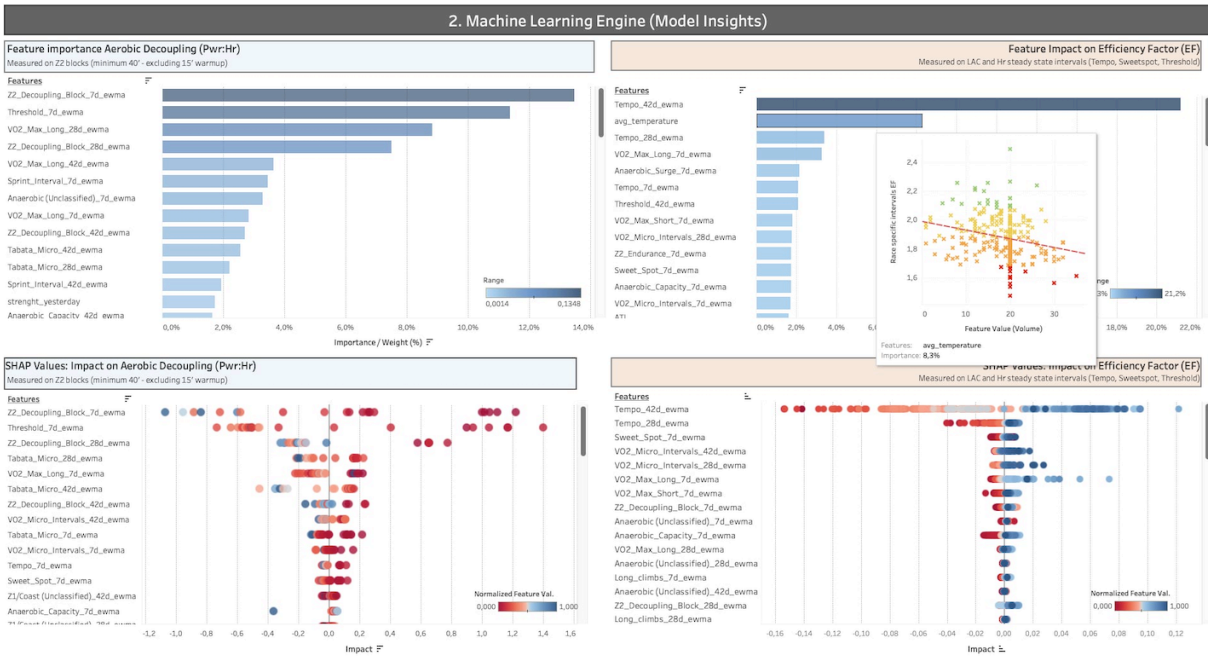


Fig. 8: Model insights layer within the Tableau dashboard, displaying feature importances and SHAP values for aerobic decoupling (left, blue) and efficiency factor (right, orange)

## 3. Aerobic Decoupling — Trend Validation

The third dashboard validates the model against observed physiological adaptation.

- Grey bars represent the **42-day EWMA of Z2 decoupling volume**
- The blue line represents **measured aerobic decoupling**

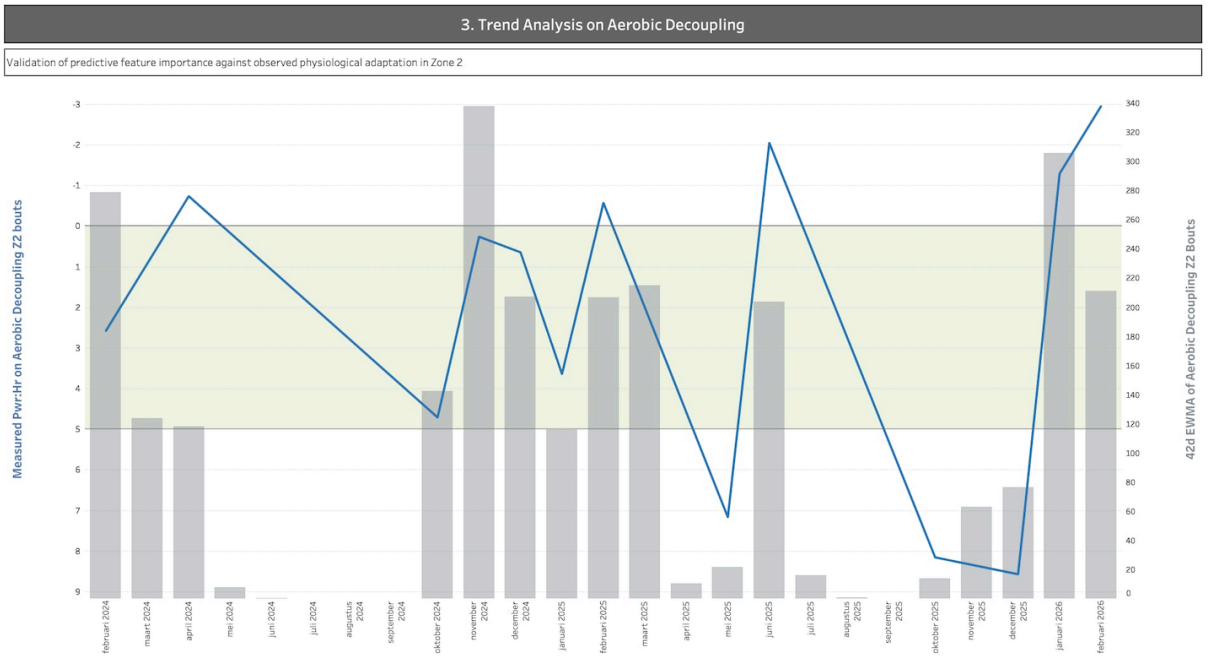


Fig. 9: Aerobic decoupling trend analysis layer within the Tableau dashboard

## Model validation (Aerobic Decoupling)

For aerobic decoupling, I validated the model using a Leave-One-Out Cross-Validation (LOOCV), given the limited number of observations (n = 22).

The results:

- MAE: 3.65
- RMSE: 5.03
- $R^2$: -0.09

On paper, this is not a strong predictive result. But in this context, that's not surprising.

Aerobic decoupling is one of the hardest metrics to model properly. The dataset is small by design, because I only included long, clean Z2 blocks. On top of that, decoupling is highly sensitive to context — fatigue, temperature, nutrition, accumulated load — which introduces a lot of variability and a few clear outliers.

With only 22 observations, LOOCV becomes unstable: each single ride has a large impact on the outcome.

So rather than focusing on absolute prediction accuracy, I looked at what the model *consistently picks up*. And there, the signal is clear. The feature importance and SHAP values remain physiologically coherent and align with domain knowledge.

In other words: the model struggles to predict exact values, but it does capture the underlying structure of what drives aerobic decoupling.

For this type of problem, that's where the real value lies.

# 4. Efficiency Factor — Longitudinal Analysis & Simulation

The fourth dashboard provides the clearest demonstration of the model's explanatory power.

The bars represent aggregated **42-day EWMA of tempo, threshold, and sweetspot volume**, while the line shows the measured **efficiency factor (EF)**.

Two distinct phases emerge:

## Phase 1 — Breakdown

Initially, EF declines despite increasing training volume.

At first glance, this could suggest a model failure. However, inspection of the interval distribution reveals the true cause:

- minimal Z2 decoupling work
- high volume of surges and anaerobic efforts
- significant time spent coasting or in fragmented intensity

In other words, the athlete accumulated load, but not the **right type of load**.

The result: reduced efficiency and a prolonged recovery period of approximately three months.

## Phase 2 — Alignment

In the second phase, training distribution shifts:

- increased Z2 and structured aerobic work
- reduced anaerobic noise
- more consistent interval execution

Here, the relationship between training load and efficiency becomes clear:

**the model prediction and measured EF move in close alignment.**

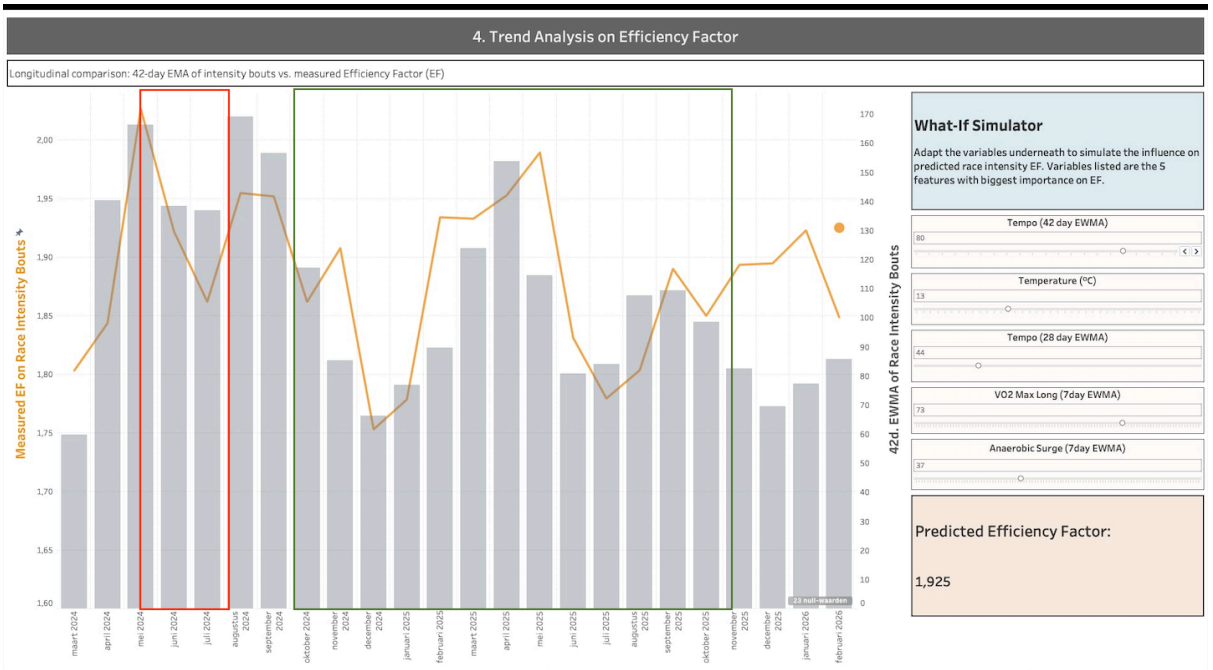


Fig. 10: EF Trend analysis layer within the Tableau dashboard. The red and green frames highlight the prolonged recovery period and the alignment phase, respectively.

## Model validation (Efficiency Factor)

To validate the model, a 5-fold cross-validation was applied.

The results:

- MAE: 0.10
- $R^2$: 0.19

At first glance, this may seem modest.

However, the efficiency factor is not a purely mechanical outcome of interval distribution. It reflects a combination of accumulated fatigue, metabolic state, environmental conditions, and execution quality — only part of which is captured in the current feature set.

What matters here is not absolute prediction accuracy, but consistency with observed behavior.

This becomes clear when comparing the two phases.

In **Phase 1**, the model detects the shift towards fragmented, surge-heavy training, but underestimates the resulting drop in efficiency.
In **Phase 2**, where training becomes more structured and controlled, predictions and observed EF move much closer together.

This pattern is consistent:
the model captures the direction and structure of change, even when it does not fully match the magnitude.

So the limitation is not that the model fails to recognize training patterns — but that efficiency remains a multi-factor outcome that cannot be fully explained by interval structure alone.

### The What-If Simulator

To extend the analysis, the dashboard includes a **what-if simulator**.

By adjusting the most important features (e.g. tempo volume, $VO_2$max work, temperature), the model dynamically updates the predicted efficiency factor.

This transforms the model from a descriptive tool into a **decision-support system**, allowing exploration of training scenarios in real time.

## Ch. 7: Limitations and road ahead (v2.0)

While this pipeline successfully translates raw training data into actionable physiological insights, it is only a Version 1.0. In the intersection of data science and sports physiology, there is always room for higher resolution. Future iterations of this project will address the following limitations:

- **The N=1 Challenge:** This model was trained on two years of data from a single athlete with a specific physiological profile (high anaerobic capacity). To validate its robustness, the pipeline is currently being tested on athletes with contrasting profiles (e.g., pure time-trialists or "diesel" engines).

- From Rule-Based to **Unsupervised Classification**: Currently, the interval classifier relies on a rule-based dictionary with hard boundaries (e.g., specific % of FTP). While

effective, human physiology operates on a **sliding scale**. V2 will implement Unsupervised Machine Learning (e.g., **K-Means** or **DBSCAN clustering**) to allow the data to organically group intervals based on multi-dimensional similarities, potentially uncovering new, hidden physiological patterns.

- **The "Averaging" Trap**: Aggregating Efficiency Factor (EF) across all tempo/sweetspot blocks in a single ride loses chronological context. A stable EF over four intervals is vastly different from an EF that crashes in the final block. Future models will incorporate **variance, minimums, and EF "slopes"** (degradation over time) rather than just daily averages.

- Dynamic Anchor Points (**eFTP**): The current model relies on a static FTP input provided by the athlete. Since FTP naturally fluctuates, an incorrect anchor point skews all derived interval classifications. Integrating a **continuously calculating "Estimated FTP"** (eFTP) based on the preceding 30-day critical power curve will make the pipeline **fully autonomous.**

- Missing **Biomechanical & Recovery Data**: The model currently evaluates the metabolic "engine" but is blind to the "chassis" (biomechanics) and the nervous system. Integrating baseline **HRV data** (parasympathetic readiness) and dual-sided power meter metrics (like **Platform Center Offset** to detect muscular fatigue or left/right asymmetries) could predict injury and performance degradation before cardiovascular metrics show any signs of distress.

- While the model aligns with domain expertise, its **criterion validity** remains limited as it hasn't been benchmarked against "gold standard" laboratory data. Comparing model outputs with **invasive lactate testing** is a necessary next step to statistically confirm that the predicted intervals accurately reflect the athlete's true physiological thresholds.

# Conclusion & Impact

Despite the current limitations, this project demonstrates that high-frequency training data can be transformed into **interpretable physiological insight** when:

- raw data is structured correctly
- intervals are classified with domain knowledge
- features reflect underlying biological processes
- models are chosen for explainability, not just performance

By mathematical analysis this pipeline highlights that training quality is not determined by volume, but by the correct distribution of physiological stimuli.

The result is not just prediction, but understanding.

And in both coaching and data science, understanding is where real progress starts.

# Final Word

This project was, above all, an eye-opener.

What started as a technical exploration quickly became something more: a way to look at training with **greater precision** and, in many ways, **greater simplicity**.

It reinforced the idea that performance is not just about doing more, but about doing the right things at the right time.

Interestingly, some of the patterns that emerged align with trends we are starting to see at the highest level. Reduced overall volume, but more controlled Zone 2 work and more targeted efforts just below threshold. Whether coincidence or not, the data seems to support a shift towards quality and specificity over sheer volume.

Athletes often say:

> *"I wish I had more time."*

This project suggests a different perspective—that time might not be the primary limitation. Structure, intent, and execution might matter more.

The same nuance applies to Zone 2. It remains essential, but this model suggests that endlessly accumulating low-intensity volume alone is not sufficient to drive improvements at race intensity. Training needs to be balanced, targeted, and context-driven.

This is only a first iteration. The model can be improved, expanded, and challenged. But if it achieves one thing, I hope it is this:

> *"That it encourages a more thoughtful, data-informed approach to training—one that ultimately leads not only to better performance, but also to more enjoyment in the process."*